

GUÍA PRÁCTICA - DESPLIEGUE DE APLICACIONES CON DOCKER COMPOSE

DESARROLLO DE APLICACIONES WEB - DAW135



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Introducción

En la práctica anterior (Semana 14), se completó con éxito el empaquetado individual de nuestros componentes, generando las imágenes locales para el backend y el frontend, además de obtener la imagen oficial de la base de datos.

Para la Semana 15, daremos el paso definitivo hacia el Despliegue de stack completo. En esta guía utilizaremos Docker Compose para definir, conectar y ejecutar de forma simultánea e integrada los tres servicios que componen nuestra arquitectura web (Frontend, Backend y Base de Datos) bajo una misma red virtual aislada.



Requerimientos Previos

Antes de comenzar con los pasos de la guía, se deben de cumplir con los siguientes requerimientos:

- Backend (Java): Imagen mi-backend:1.0.
- Frontend (React): Imagen mi-frontend:1.0.
- Base de Datos: Imagen oficial postgres:16-alpine.

Ubicación y Ruta del Archivo de Orquestación

Para lograr una administración limpia y centralizada de toda la infraestructura, se debe ubicar el archivo *docker-compose.yml* en la raíz del espacio de trabajo. De esta manera, se sitúa al mismo nivel jerárquico que las carpetas de desarrollo de cada aplicación.

Ruta de archivos

Navegar en el explorador de archivos o la terminal hacia la carpeta del proyecto (*mi-proyecto/*). Dependiendo del sistema operativo y de cómo se hayan organizado las carpetas, la ruta podría verse similar a una de las siguientes opciones:

- Alternativa en Windows:
C:\Usuarios\<tu_usuario>\Documentos\mi-proyecto
- Alternativa en macOS / Linux:
*/home/<tu_usuario>/Documentos/mi-proyecto/ o utilizando el atajo
~/Documentos/mi-proyecto/*

Estructura del proyecto final

Se debe verificar que el proyecto en Visual Studio Code refleje exactamente la siguiente organización:

```
None
mi-proyecto/
|
├── mi-backend/
│   ├── src/
│   ├── target/
│   ├── Dockerfile
│   ├── pom.xml
│   ├── mvnw
│   └── mvnw.cmd
|
├── mi-frontend/
│   ├── src/
│   ├── public/
│   ├── dist/
│   ├── Dockerfile
│   ├── package.json
│   ├── package-lock.json
│   ├── index.html
│   └── vite.config.js
|
└── docker-compose.yml
```

Creación y Configuración del Docker-Compose

En la carpeta raíz *mi-proyecto/*. Se debe crear un archivo nuevo con el nombre exacto *docker-compose.yml*.

Se debe agregar el siguiente bloque de código. Se debe tener en cuenta que los parámetros de conexión de PostgreSQL se sincronizan de forma estricta con las propiedades fijadas en el archivo *application.properties* del backend (midb, postgres, tu_password).

```
None
version: '3.8'

services:
  # 1. Servicio de Base de Datos (PostgreSQL)
  database:
```

```
image: postgres:16-alpine
container_name: app_database
restart: always
environment:
  POSTGRES_USER: postgres
  POSTGRES_PASSWORD: tu_password
  POSTGRES_DB: midb
ports:
  - "5432:5432"
volumes:
  - postgres_data:/var/lib/postgresql/data
networks:
  - app_network
```

2. Servicio Backend (Java / Spring Boot)

```
backend:
  image: mi-backend:1.0
  container_name: app_backend
  restart: always
  ports:
    - "8080:8080"
  environment:
    - SPRING_DATASOURCE_URL=jdbc:postgresql://database:5432/midb
    - SPRING_DATASOURCE_USERNAME=postgres
    - SPRING_DATASOURCE_PASSWORD=tu_password
  depends_on:
    - database
  networks:
    - app_network
```

3. Servicio Frontend (React / Nginx)

```
frontend:
  image: mi-frontend:1.0
  container_name: app_frontend
  restart: always
  ports:
    - "80:80"
  depends_on:
    - backend
  networks:
    - app_network
```

Volumen para garantizar que la información de las tablas no se borre

```
volumes:
  postgres_data:
```

Red interna privada para intercomunicar los contenedores

```
networks:
  app_network:
    driver: bridge
```

Descripción de propiedades utilizadas en Docker-Compose

Propiedad	Descripción
container_name	Asigna un nombre fijo y legible al contenedor en ejecución.
ports	Vincula un puerto de la computadora real con el puerto interno del contenedor (por ejemplo, 8080:8080 para Spring Boot y 80:80 para el servidor HTTP Nginx del Frontend).
depends_on	Define la jerarquía y secuencia de encendido. Evita que el backend inicie antes de que la base de datos esté lista para aceptar conexiones o que el frontend inicie antes que el backend.
volumes	Vincula un espacio del disco duro local (postgres_data) con la ruta interna de datos de PostgreSQL, guardando la información registrada.
networks	Agrupar los servicios en una red aislada (app_network). Permite al backend invocar la URL de base de datos usando el alias de red <code>jdbc:postgresql://database:5432/midb</code> en vez de una IP variable.

Despliegue de Proyecto y Contenedores

Se debe abrir la terminal y asegurarse de estar posicionados en la raíz del proyecto.

1. Ubicarse en la raíz del proyecto:

Shell

Windows

```
cd C:\Usuarios\<tu_usuario>\Documentos\mi-proyecto\
```

Linux y MacOS

```
cd /home/<tu_usuario>/Documentos/mi-proyecto/
```

2. Lanzar e inicializar el Stack Completo:

Se debe ejecutar el comando agregando el parámetro `-d` (detached mode). Esto levantará toda la infraestructura en segundo plano, liberando la terminal para seguir trabajando.

Shell

```
docker-compose up -d
```

Al ejecutarlo, la pantalla mostrará la creación ordenada de la red virtual, el volumen de datos, y el inicio sucesivo de *app_database*, *app_backend* y *app_frontend*.

```
PS C:\Users\cnava\Downloads\MI-PROYECTO> docker-compose up -d
[+] Running 5/5
✓ Network mi-proyecto_app_network      Created
0.4s
✓ Volume "mi-proyecto_postgres_data"  Created
0.0s
✓ Container app_database               Started
1.4s
✓ Container app_backend                Started
1.8s
✓ Container app_frontend               Started
2.2s
```

Verificación y Validación del Despliegue

Para validar que las imágenes interactúan perfectamente entre sí dentro del ecosistema orquestado, se deben realizar los siguientes pasos de control:

- Validación en CLI: Al escribir `docker-compose ps`, la columna **STATUS** de los tres contenedores debe permanecer estable en **Up**.

```
PS C:\Users\cnava\Downloads\MI-PROYECTO> docker-compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED      STATUS      PORTS
app_backend         mi-backend:1.0      "java -jar app.jar"    backend    19 seconds ago Up 14 seconds 0.0.0.0:8080->8080/tcp
app_database        postgres:16-alpine   "docker-entrypoint.s..." database    19 seconds ago Up 15 seconds 0.0.0.0:5432->5432/tcp
app_frontend        mi-frontend:1.0     "/docker-entrypoint..." frontend    19 seconds ago Up 14 seconds 0.0.0.0:80->80/tcp
```

- Validación desde Docker Desktop: Se debe abrir la interfaz gráfica de Docker Desktop, navegar a la sección Containers y corroborar que aparezca un grupo colapsable llamado **mi-proyecto** conteniendo los tres iconos en color verde encendido.

Containers
Give feedback
View all your running containers and applications. Learn more

Container CPU usage
0.20% / 1600% (16 CPUs available)

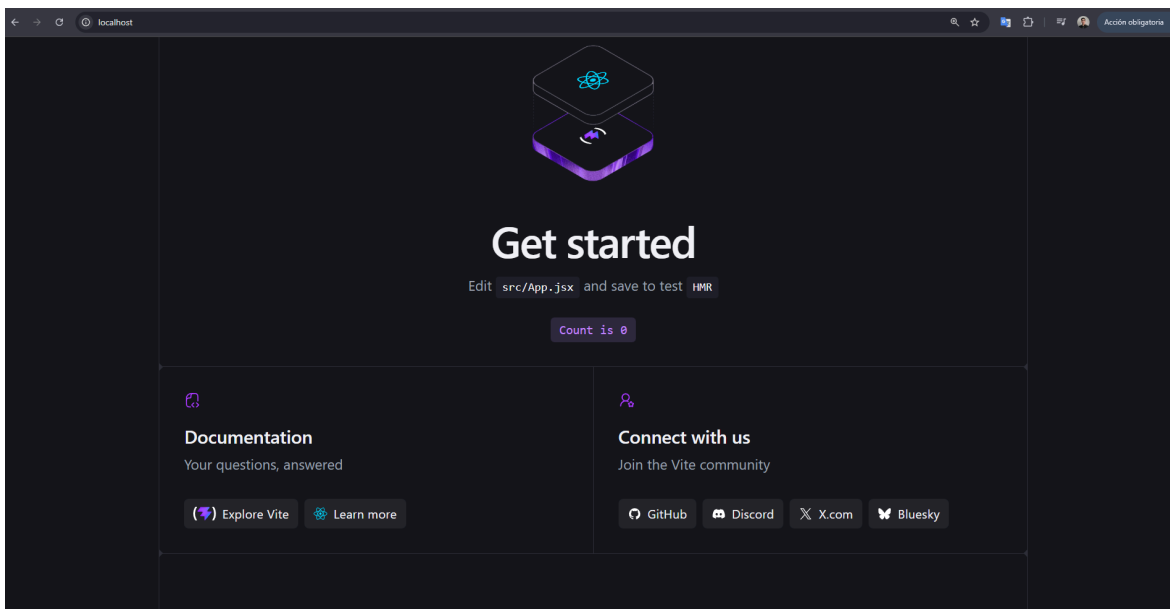
Container memory usage
260.63MB / 7.26GB

Show charts

☐ Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	mi-proyecto	-	-	-	0.2%	58 seconds ago	
☐	app_database	44e1124b6e83	postgres:16-alpine	5432:5432	0.03%	59 seconds ago	
☐	app_backend	4d4b843921d4	mi-backend:1.0	8080:8080	0.17%	58 seconds ago	
☐	app_frontend	1b93a69f0ab4	mi-frontend:1.0	80:80	0%	58 seconds ago	

- Comprobación en el Navegador: Se debe abrir el navegador preferido e ingresar a <http://localhost>. El puerto 80 mostrará la pantalla de bienvenida o la interfaz de usuario de React servida por Nginx, la cual ahora tiene acceso transparente hacia la API del backend en el puerto 8080.



Limpieza de Contenedores

Cuando se finalizó la sesión práctica, se pueden destruir los contenedores de forma segura. El siguiente comando remueve los contenedores y limpia las redes virtuales creadas sin tocar los datos guardados en la base de datos:

```
Shell  
docker-compose down
```

Conclusión

A través de la presente guía, se han evidenciado las ventajas operativas de la infraestructura como código (IaC) empleando Docker Compose. Al unificar múltiples servicios en un único manifiesto de configuración centralizado, se logró abstraer la complejidad de las redes virtuales y la persistencia de datos, asegurando que un sistema multicapa complejo pueda levantarse sin variaciones o configuraciones manuales propensas a fallos en cualquier máquina cliente o de producción.

